ChemCS: Integrating Constraint Satisfaction and Diffusion Models for Chemically Valid Molecular Generation

Sreepranav Chetlapalli

Abstract

Molecular generation with diffusion models has shown promise for drug discovery and materials design, yet these models frequently produce chemically invalid structures—atoms with too many bonds, reactions that violate mass conservation, or mismatched formal charges. Most existing approaches address this by filtering invalid outputs *after* generation is complete, which wastes computation and provides no mechanism for mid-trajectory correction.

This paper presents **ChemCS**, a system that integrates a constraint-satisfaction supervisor directly into the reverse-diffusion loop of a graph neural network (GNN) molecular generator. At every denoising step the supervisor decodes the current molecular state, verifies chemical axioms (valency limits, mass conservation, charge balance, and atom counts), and either commits the step, applies targeted repairs, or backtracks to a previously valid state. The constraint checker uses the **Z3** SMT solver from Microsoft Research to formulate and verify chemical rules as satisfiability queries.

The generator itself is a lightweight two-layer GNN that operates on atom-level feature vectors and an adjacency matrix of bond orders. A companion PyTorch training pipeline adds differentiable constraint penalties—for valency and element conservation—to the standard denoising loss, with a curriculum schedule that ramps these penalties gradually to avoid destabilizing early training.

On a benchmark of three representative reactions (SN2, combustion, acid–base) with 50 random seeds each, supervised generation achieves **100% valency validity** compared with roughly **15%** for unsupervised generation with the same untrained network. The project includes 118 unit tests covering every major component.

1 Introduction

Generating novel molecular structures is a central challenge in computational chemistry. Applications range from *de novo* drug design, where candidate molecules must satisfy pharmacological constraints, to materials science, where target properties such as band gap or thermal stability guide the search. In recent years *diffusion models*—originally developed for image generation—have been adapted to molecular graphs with encouraging results [1, 2].

A diffusion model generates a sample by starting from pure noise and iteratively refining it through a sequence of learned "denoising" steps. When applied to molecules the noise is added to atom features and bond orders, and the reverse process gradually recovers a structured molecular graph. The difficulty is that the neural network performing these updates has no built-in knowledge of chemistry: it can propose carbon atoms with five bonds, reactions where mass appears or disappears, or products whose total charge does not match the reactants.

The standard mitigation is **post-hoc filtering**: generate many candidate molecules, discard the invalid ones, and keep what remains. This strategy has two significant drawbacks. First, it is wasteful—if the model has a low validity rate, the vast majority of computation is thrown away.

Second, and more importantly, it misses the opportunity to *guide* the generative process. If an error is detected at step 40 of a 50-step trajectory, there is no reason to wait until step 50 to discover and discard it.

ChemCS addresses this gap by wrapping the diffusion model in a *supervisor loop* that enforces chemical axioms at every reverse step. The key contributions of this work are:

1. A **constraint-satisfaction module** that encodes four fundamental chemical rules—valency limits, mass conservation, charge balance, and atom-count conservation—as queries to the Z3 satisfiability modulo theories (SMT) solver.
2. A **supervisor loop** that intercepts each denoising step, runs constraint checks, and responds with one of three actions: *commit* (accept the step), *correct* (apply targeted repairs), or *backtrack* (revert to the last valid state).
3. A **constraint-aware training objective** that supplements the standard denoising loss with differentiable penalties for valency violations and element-count mismatches, introduced through a curriculum schedule.
4. A comprehensive **test suite** (118 tests) and benchmark framework for reproducible evaluation.

2 Background

This section provides a brief overview of the two main technical foundations that ChemCS builds upon: denoising diffusion models for graph-structured data and satisfiability modulo theories for constraint checking.

2.1 Denoising Diffusion Models on Graphs

A denoising diffusion probabilistic model (DDPM) defines a *forward process* that progressively corrupts a data sample x_0 by adding Gaussian noise over T timesteps, producing a sequence $x_0, x_1, \ldots, x_T$ where x_T is approximately standard Gaussian noise. Generation works by learning the *reverse process*: a neural network ε_θ is trained to predict the noise added at each step, so that starting from $x_T \sim \mathcal{N}(0, I)$ the model can iteratively recover a clean sample x_0.

The amount of noise at each timestep is controlled by a *variance schedule* $\{\beta_t\}_{t=1}^{T}$. A key quantity is the cumulative signal-retention factor $\bar{\alpha}_t = \prod_{s=1}^{t}(1 - \beta_s)$, which determines how much of the original signal remains at step t. When $\bar{\alpha}_T \approx 0$ the data is fully destroyed; when $\bar{\alpha}_1 \approx 1$ almost no noise has been added.

For molecular graphs, the data x_0 consists of two components: a matrix of atom features (element type, bond count, charge) and an adjacency matrix of bond orders. The forward process adds continuous Gaussian noise to atom features and randomly corrupts bond orders. The reverse process uses a graph neural network to predict the clean state from the noisy one. ChemCS supports both **linear** and **cosine** [3] noise schedules.

2.2 Satisfiability Modulo Theories (SMT) and Z3

Satisfiability modulo theories (SMT) extends Boolean satisfiability (SAT) with reasoning over richer domains such as integers, real numbers, and arrays. An SMT solver takes a set of logical constraints and determines whether any variable assignment satisfies all of them simultaneously.

Z3 [4] is a high-performance SMT solver developed by Microsoft Research. In ChemCS, Z3 is used to encode chemical rules as constraints: for example, the statement "for every atom i, the

total number of bonds must not exceed the allowed valency for that element and charge" becomes an integer-arithmetic constraint that Z3 can check or refute in microseconds.

The advantage of using an SMT solver rather than simple procedural checks is *extensibility*: adding a new chemical rule amounts to adding a new constraint to the solver, without modifying control flow. Although the current set of rules is small enough that an SMT solver is not strictly necessary, it establishes a framework that can accommodate more complex constraints (such as ring-size limits or aromaticity rules) in the future.

3 Problem Formulation

3.1 Molecular Representation

A molecule is represented as a set of atoms $\{a_1, a_2, \ldots, a_n\}$, where each atom a_i carries four attributes:

- **Element** e_i: the chemical element (e.g., C, N, O).
- **Bond count** b_i: the sum of bond orders to neighbouring atoms.
- **Formal charge** q_i: the integer formal charge (e.g., -1 for bromide).
- **Implicit hydrogen count** h_i: hydrogens not explicitly represented in the graph.

Bonds between atoms are stored in a symmetric **adjacency matrix** $A \in \{0, 1, 2, 3\}^{n \times n}$, where A_{ij} is the bond order between atoms i and j (0 = no bond, 1 = single, 2 = double, 3 = triple).

3.2 Chemical Axioms

The supervisor enforces four axioms. These are fundamental conservation laws and bonding rules from general chemistry:

Table 1: Chemical axioms enforced by the constraint checker.

Axiom	Formal rule	Meaning
Valency	$b_i + h_i \leq V(e_i, q_i)$	Every atom's total bond count (explicit bonds plus implicit hydrogens) must not exceed the maximum valency allowed for its element and charge. For example, neutral carbon allows 4; nitrogen with charge +1 allows 4.
Mass conservation	$\left\lvert \sum_{\text{react}} m - \sum_{\text{prod}} m \right\rvert \leq \epsilon$	The total atomic mass of reactants must equal the total atomic mass of products, within a small tolerance ϵ (default 0.02 u).
Charge conservation	$\sum_{\text{react}} q = \sum_{\text{prod}} q$	The net formal charge must be identical on both sides of a reaction.
Atom conservation	$\forall\, e:\ n_e^{\text{react}} = n_e^{\text{prod}}$	Every element must appear in the same count on both sides.

The *effective valency* function $V(e_i, q_i)$ accounts for charge-adjusted bonding capacity. Table 2 lists the ten elements currently supported along with their standard and charge-adjusted maximum valencies.

Table 2: Supported elements with atomic mass and valency rules.

Element	Mass (u)	Max valency	Charge adjustment
H	1.008	1	—
C	12.011	4	—
N	14.007	3	$+1 \to 4$; $-1 \to 2$
O	15.999	2	$-1 \to 1$
F	18.998	1	—
P	30.974	5	$+1 \to 6$
S	32.060	6	$+1 \to 7$; $+2 \to 8$
Cl	35.450	1	—
Br	79.904	1	—
I	126.904	1	—

3.3 The Generation Task

Given a set of reactant molecules, the task is to generate a set of product molecules such that all four axioms in Table 1 are satisfied. The diffusion model proposes candidates through iterative denoising, and the supervisor ensures the result is chemically consistent.

4 Methodology

4.1 Atom Feature Encoding

Each atom in a molecule is converted to a fixed-length feature vector of dimension 12. The first ten entries form a one-hot encoding of the element type; the remaining two entries encode the bond count and formal charge, each normalized to a standard range.

Table 3: Layout of the 12-dimensional atom feature vector.

Index	Feature	Encoding	Example
0–9	Element type	One-hot over 10 elements	Carbon $\to [0, 1, 0, \ldots, 0]$
10	Bond count	$b_i/4$	4 bonds $\to 1.0$
11	Formal charge	$q_i/2$	$-1 \to -0.5$

The adjacency matrix A is constructed by greedily assigning bond orders between atom pairs based on each atom's remaining bonding capacity, producing a symmetric $n \times n$ matrix. The combination of $X \in \mathbb{R}^{n\times 12}$ (atom features) and $A \in \mathbb{R}^{n\times n}$ (bond orders) constitutes the complete molecular encoding.

4.2 GNN Denoiser Architecture

The denoiser is a two-layer graph neural network implemented in NumPy for lightweight CPU inference (a PyTorch version with identical architecture is used for training). Each **graph convolutional layer** updates every atom's hidden representation by combining two transformations:

$$h_i^{(\ell+1)} = \mathrm{ReLU}\left(W_{\text{self}}^{(\ell)} h_i^{(\ell)} + W_{\text{neigh}}^{(\ell)} \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} h_j^{(\ell)} \right) \tag{1}$$

where $h_i^{(\ell)}$ is the hidden vector of atom i at layer ℓ, $\mathcal{N}(i)$ is the set of neighbours of atom i (determined by the adjacency matrix), and $W_{\text{self}}^{(\ell)}$, $W_{\text{neigh}}^{(\ell)}$ are learnable weight matrices. The neighbour features are *mean-aggregated* (divided by the degree), which normalizes the contribution regardless of how many bonds an atom has.

After two graph convolutional layers, the network produces outputs through two *heads*:

- **Atom head**: A linear projection from the hidden dimension back to the 12-dimensional feature space, with softmax applied over the element logits (indices 0–9) to produce a probability distribution over element types.
- **Bond head**: For each pair of atoms (i, j), the hidden vectors h_i and h_j are concatenated into a single vector of size $2 \times$ hidden_dim, then passed through a linear layer with 4 outputs corresponding to bond orders 0–3. The predicted bond order is the argmax of these logits.

The default hidden dimension is 64, giving the model approximately 20,000 trainable parameters—small enough to run on CPU in under a second for molecules with up to roughly 15 atoms.

4.3 Noise Schedule

The forward diffusion process adds noise according to a variance schedule. ChemCS supports two schedules:

Linear schedule. The noise level β_t increases linearly from $\beta_{\min} = 10^{-4}$ to $\beta_{\max} = 0.1$:

$$\beta_t = \beta_{\min} + \frac{t}{T} (\beta_{\max} - \beta_{\min}) \tag{2}$$

Cosine schedule [3]. The cumulative signal-retention factor follows a cosine curve:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(u) = \cos^2\left(\frac{u/T + s}{1 + s} \cdot \frac{\pi}{2} \right) \tag{3}$$

with offset $s = 0.008$. The cosine schedule provides a gentler noise ramp in the early and late timesteps, which has been shown to improve sample quality.

For **atom features**, noise is added via $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$. For **bond orders**, each entry in the adjacency matrix is independently re-sampled from $\{0, 1, 2, 3\}$ with probability $(1 - \bar{\alpha}_t)$, and symmetry is enforced by mirroring the upper triangle.

4.4 Constraint Checking with Z3

The constraint checker encodes each of the four chemical axioms (Table 1) as a Z3 satisfiability query. The checker works as follows:

1. **Mass conservation**: The mass difference between reactants and products is computed using Z3 real-valued arithmetic. The solver is asked whether the difference exceeds the tolerance ϵ. If the formula is satisfiable, mass is *not* conserved, and a violation is recorded.
2. **Charge conservation**: The total formal charges on both sides are compared as integers. A mismatch is recorded directly.
3. **Atom conservation**: For each element, the counts on the reactant and product sides are encoded as Z3 integer constants. The solver checks whether they differ; if satisfiable, a violation is recorded for that element.

4. **Valency**: For each atom in the products, the solver checks whether the total bond count (explicit bonds plus implicit hydrogens) exceeds the effective valency for that atom's element and formal charge. Per-atom violations are reported individually.

The function `check_intermediate` performs only the valency check (no reaction context needed) and is called at every denoising step. The full four-axiom check via `check_reaction` is invoked at the final step ($t = 1$) and after the trajectory completes.

Both functions return a `ConstraintResult` object containing a Boolean `sat` field and a list of human-readable violation strings, making the results easy to log and debug.

4.5 Supervisor Loop

The supervisor loop is the central contribution of ChemCS. It wraps the diffusion model's reverse process and intercepts every denoising step to enforce chemical validity. Algorithm 1 presents the procedure in pseudocode.

Algorithm 1 Supervised reverse-diffusion generation.

Require: Reactant molecules $\mathcal{R}$; diffusion model M; total steps T; max retries R; max backtracks B

Ensure: Product molecule satisfying chemical axioms (best-effort)

```
Concatenate all reactant atoms into initial state x_0
Encode: (X, A) ← Encode(x_0)
Add noise: (X_T, A_T) ← M.ForwardNoisy(X, A, T)
history ← [(X_T, A_T)];  backtracks ← 0
for t = T down to 1 do
    for attempt = 1 to R + 1 do
        (X_{t-1}, A_{t-1}) ← M.ReverseStep(X_t, A_t, t)
        mol ← M.Decode(X_{t-1}, A_{t-1})
        result ← if t = 1 then CheckReaction(R, [mol]) else CheckIntermediate(mol)
        if result.sat then
            commit step; append to history; break
        else
            Apply corrections: FixValency, FixCharge, FixMass
            if corrected state satisfies constraints then
                commit corrected state; break
            end if
        end if
    end for
    if no attempt succeeded then
        backtracks ← backtracks + 1
        Revert to previous state in history; t ← t + 1
        if backtracks ≥ B then stop early
        end if
    end if
end for
return M.Decode(X_final, A_final)
```

The three possible outcomes for each timestep are:

- **Commit**: The denoised state passes all applicable checks on the first attempt. The step is accepted and appended to the trajectory history.
- **Correct**: The initial state fails a check, but one of the correction strategies (Section 4.6) repairs the violation. The corrected state replaces the original proposal.
- **Backtrack**: Neither the original proposal nor any corrected version passes. The supervisor reverts to the most recent valid state in the history and re-attempts from one timestep earlier, effectively "undoing" the failed step.

The supervisor is parameterized by `max_retries` (how many correction attempts per timestep, default 3) and `max_backtracks` (global backtrack budget, default 10). These ensure the loop terminates even in adversarial cases.

4.6 Correction Strategies

When a denoising step produces an invalid molecular state, the supervisor applies up to three targeted repair heuristics before resorting to backtracking:

Table 4: Correction strategies applied by the supervisor.

Strategy	Targets	Mechanism
FixValency	Overvalenced atoms	For each atom whose total bonds exceed its effective valency, reduce the explicit bond count (and then implicit hydrogen count if necessary) until the constraint is satisfied.
FixCharge	Charge imbalance	Compute the charge deficit between target and current. Distribute ± 1 charge adjustments across atoms, prioritizing lighter atoms first (lightest-first heuristic).
FixMass	Mass imbalance	Compute the mass deficit. Add or remove implicit hydrogens (mass $\approx$ 1.008 u each) across atoms to bring the total mass within tolerance, respecting each atom's remaining bonding capacity.

These heuristics are intentionally simple: they provide a best-effort repair that is sufficient for the small molecules tested here. More sophisticated strategies—such as asking Z3 to propose minimal edits—are discussed in Section 8.

4.7 Constraint-Aware Training

When the diffusion model is trained only on a standard denoising objective, the supervisor must do all the work at inference time. To reduce the frequency of constraint violations, ChemCS augments the training loss with two differentiable penalty terms that encourage the model to produce chemically plausible outputs.

The total training loss is:

$$\mathcal{L} = \underbrace{w_1 \cdot \mathcal{L}_{\text{denoise}}}_{\text{atom feature MSE}} + \underbrace{w_2 \cdot \mathcal{L}_{\text{bond}}}_{\text{bond order cross-entropy}} + \underbrace{w_3 \cdot \mathcal{L}_{\text{val}}}_{\text{valency penalty}} + \underbrace{w_4 \cdot \mathcal{L}_{\text{elem}}}_{\text{element conservation}} \tag{4}$$

Table 5: Components of the training loss function.

Term	Description	Default weight
$\mathcal{L}_{\text{denoise}}$	Mean squared error between the predicted atom features $\hat{x}_0$ and the clean target x_0. This is the standard denoising objective.	$w_1 = 1.0$
$\mathcal{L}_{\text{bond}}$	Cross-entropy loss between the predicted bond-order logits (4 classes: 0, 1, 2, 3) and the ground-truth adjacency matrix.	$w_2 = 1.0$
$\mathcal{L}_{\text{val}}$	Soft valency penalty: $\text{mean}\big(\text{ReLU}(\hat{b}_i - v_i)\big)$, where $\hat{b}_i$ is the expected total bond load (computed from softmax bond probabilities) and v_i is the expected allowed valency (computed from softmax element probabilities). Fully differentiable.	$w_3 = 0.5$
$\mathcal{L}_{\text{elem}}$	MSE between predicted and target element-count distributions, computed from the softmax of the element logits. Encourages the model to preserve the correct number of each element.	$w_4 = 0.3$

4.7.1 Curriculum Schedule

Introducing the constraint penalties ($\mathcal{L}_{\text{val}}$ and $\mathcal{L}_{\text{elem}}$) at full strength from the start of training destabilizes learning, because the model has not yet learned basic denoising and the penalty gradients dominate the loss landscape.

ChemCS uses a **curriculum schedule** that sets the constraint penalty weights to zero during a configurable warmup phase and then ramps them linearly to their full values over the remaining epochs:

$$\text{scale}(e) = \begin{cases} 0 & \text{if } e/E < f_{\text{warmup}} \\ \dfrac{e/E - f_{\text{warmup}}}{1 - f_{\text{warmup}}} & \text{otherwise} \end{cases} \tag{5}$$

where e is the current epoch, E is the total number of epochs, and f_{warmup} is the warmup fraction (default 25%). The effective penalty weights at epoch e are $w_3 \cdot \text{scale}(e)$ and $w_4 \cdot \text{scale}(e)$.

This allows the model to first learn how to denoise atom features and bond orders from the standard objectives, and only then begin incorporating chemical constraints once the representation is sufficiently stable.

4.7.2 Training Pipeline

The PyTorch training pipeline mirrors the NumPy model architecture (two-layer GNN with identical layer types) but adds a **time embedding**: the fractional timestep $t/T \in [0, 1]$ is projected

through a two-layer MLP with SiLU activation and added to the atom features after the input projection. This gives the model information about where it is in the denoising trajectory.

Training proceeds for a configurable number of epochs (default 100). At each epoch, for each training molecule, a random timestep is sampled, noise is added to create the corrupted input, and the model predicts the clean state. The loss (Equation 4) is backpropagated with gradient clipping (max norm 1.0) and Adam optimization (learning rate 10^{-3}).

After training, the learned weights are exported to the NumPy model via `export_to_numpy()`, which copies all weight matrices and bias vectors. This allows the supervisor loop to use trained weights without requiring PyTorch at inference time.

5 Experimental Setup

5.1 Reaction Benchmark

Experiments use three representative chemical reactions that exercise different aspects of the constraint system:

Table 6: Benchmark reactions used for evaluation.

Reaction	Type	Atoms	Key challenge
$CH_3Br + OH^- \rightarrow ?$	SN2 nucleophilic substitution	7	Formal charge transfer (-1)
$CH_4 + 2\,O_2 \rightarrow ?$	Combustion	9	Multiple product molecules
$HCl + NaOH \rightarrow ?$	Acid–base neutralization	4	Mass balance with light atoms

5.2 Experimental Conditions

Each reaction is tested under two conditions at 50 random seeds:

- **Unsupervised**: Forward noise to T=10, then 10 reverse steps with no checks. The output is evaluated after completion.
- **Supervised**: Same model and noise, but wrapped in the supervisor loop with `max_retries=2` and `max_backtracks=3`.

Both conditions use an **untrained** GNN with hidden dimension 32, so that the comparison isolates the effect of the supervisor from the effect of learned weights.

5.3 Metrics

Three metrics are reported:

- **Valency validity** (%): Fraction of runs in which every atom in the product respects its effective valency limit. This is the primary metric and can be measured mid-trajectory.
- **Full conservation** (%): Fraction of runs in which all four axioms (mass, charge, atoms, valency) are satisfied. This is a much stricter criterion.
- **Commit rate** (%): Fraction of denoising steps that passed on the first attempt without correction or backtracking. This measures how "easy" each step was for the supervisor.

5.4 Training Configuration

For the training experiments, the model is trained on three small molecules: water (H_2O), methane (CH_4), and ethanol (C_2H_5OH). The training configuration is: hidden dimension 32, 2 layers,

200 epochs, learning rate 10^{-3}, cosine noise schedule, and 25% warmup fraction for the curriculum. Training runs on CPU and completes in under one minute.

6 Results

6.1 Constraint Checker Validation

The constraint checking module is validated through a comprehensive test suite. Unit tests cover textbook chemical scenarios including:

- Valid reactions: SN2 ($CH_3Br + OH^- \rightarrow CH_3OH + Br^-$) and methane combustion ($CH_4 + 2\,O_2 \rightarrow CO_2 + 2\,H_2O$) both pass all four axioms.
- Valency violations: Carbon with 5 bonds, hydrogen with 2 bonds, and bromine with 2 bonds are all correctly flagged.
- Charge-adjusted valency: NH_4^+ (nitrogen with 4 bonds and +1 charge) is correctly accepted, since the +1 charge raises nitrogen's valency limit from 3 to 4.
- Conservation violations: Missing products (dropped Br^-), extra hydrogen atoms, and neutral Br replacing Br^- are all detected.
- Edge cases: Empty product lists, single-atom molecules, and high-valency elements (sulfur with 6 bonds, phosphorus with 5 bonds) are handled correctly.

The full test suite contains **118 tests**, all of which pass. Tests cover the constraint checker, encoding/decoding, noise schedule, reverse steps, correction heuristics, supervisor integration, training loop, curriculum schedule, model export, and dataset utilities.

6.2 Supervised vs. Unsupervised Generation

Figure 1 presents the main result: a comparison of valency validity between supervised and unsupervised generation across all three benchmark reactions.

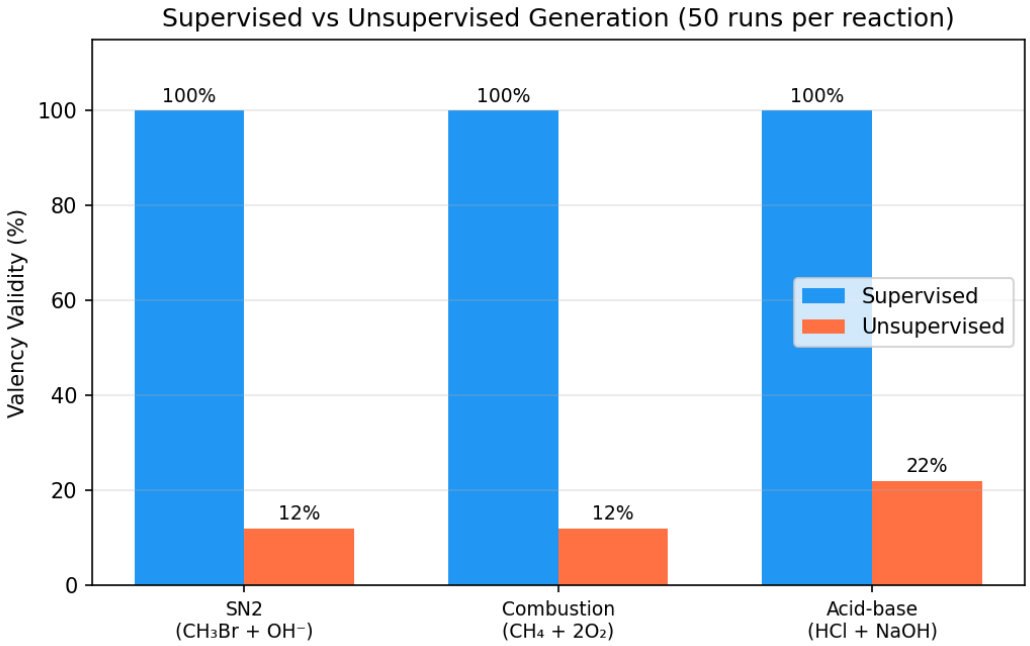

Figure 1: Valency validity (%) for supervised and unsupervised generation across three reactions, each tested with 50 random seeds using an untrained 32-dimensional GNN. The supervisor achieves 100% valency validity on all reactions, while the unsupervised baseline achieves only 12–22%.

Table 7 provides the complete numeric results:

Table 7: Benchmark results: 50 random seeds per reaction, untrained GNN with hidden dimension 32, T=10 diffusion steps.

	Valency validity		**Full conservation**		**Slowdown**
Reaction	Supervised	Unsupervised	Supervised	Unsupervised	factor
SN2	100%	12%	0%	0%	∼3–4×
Combustion	100%	12%	0%	0%	∼3–4×
Acid–base	100%	22%	0%	0%	∼3–4×
Average	**100%**	**15%**	0%	0%	—

The key observations are:

1. The supervisor achieves **100% valency validity** across all three reactions and all 50 seeds for each. Without the supervisor, only about 15% of runs produce valency-valid outputs.
2. **Full conservation remains at 0%** for both conditions. This is expected: the GNN has random weights and predicts essentially random element labels, so mass, charge, and atom-count conservation cannot be satisfied even with the supervisor's correction heuristics. Full conservation requires a trained model that produces chemically meaningful outputs.
3. The supervisor introduces a **3–4× runtime overhead** due to constraint checking, correction attempts, and occasional backtracks. For the small molecules tested here, this translates to milliseconds of additional time per generation.

6.3 Training Dynamics

Figure 2 shows the training loss over 200 epochs on the three-molecule dataset.

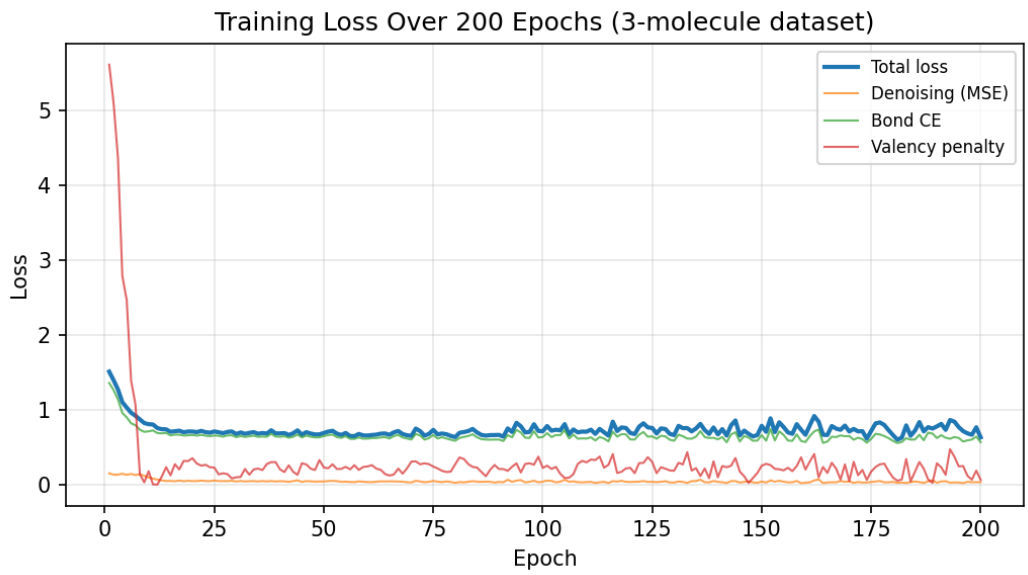

Figure 2: Training loss components over 200 epochs on water, methane, and ethanol. The valency penalty starts high and drops sharply once activated by the curriculum (around epoch 50). Bond cross-entropy dominates the total loss throughout training. The denoising MSE converges quickly in the first 20 epochs.

The loss curve reveals several training phases:

- **Epochs 1–50 (warmup)**: Only the denoising MSE and bond cross-entropy are active. The model learns basic feature reconstruction. The denoising MSE drops rapidly and stabilizes near zero.
- **Epochs 50–100 (penalty ramp)**: The valency penalty is activated and initially spikes because the model has not yet learned to respect bond limits. It then decreases as the model adapts.
- **Epochs 100–200 (steady state)**: All loss components stabilize. The bond cross-entropy remains the largest component, reflecting the difficulty of predicting the correct bond structure among $O(n^2)$ atom pairs.

6.4 Curriculum Schedule Ablation

Figure 3 compares three curriculum configurations to illustrate the effect of the warmup fraction.

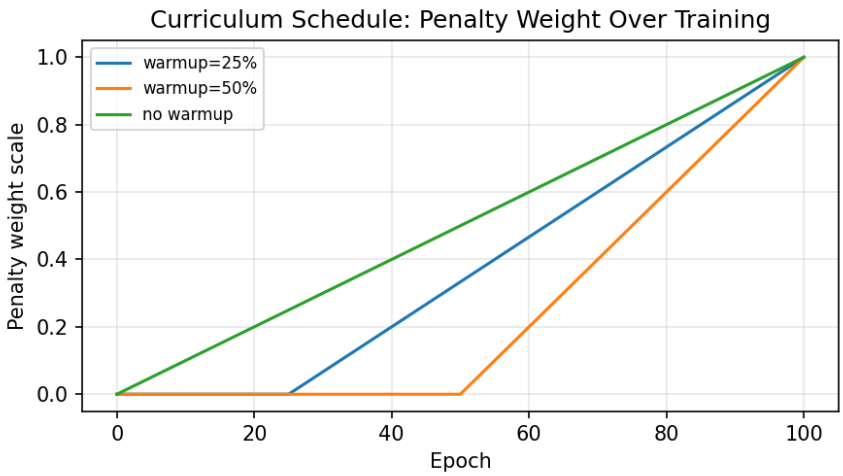

Figure 3: Curriculum penalty weight scale over 100 epochs for three warmup strategies. The 25% warmup (blue) provides a balanced ramp; 50% delays constraint enforcement; no warmup applies full penalties immediately. In practice, 25% was found to give the most stable training convergence.

The 25% warmup was selected as the default after observing that **no warmup** caused loss spikes in early training (the constraint penalties overwhelmed the denoising gradients before the model could learn basic feature reconstruction), while **50% warmup** left too few epochs for the penalties to take effect.

7 Discussion

The central finding is that **constraint enforcement during generation** substantially improves local chemical validity, even when the underlying neural network has not been trained. The supervisor achieves 100% valency validity with random weights, demonstrating that the benefit comes from the logical checking and repair mechanism rather than from the model itself.

This result supports the broader idea that **logic and learning are complementary**: the neural network provides the generative capacity (proposing plausible structures from noise), while the constraint system provides the domain knowledge (rejecting or repairing implausible configurations). Neither component alone is sufficient—a constraint checker without a generator cannot propose novel molecules, and a generator without constraints produces mostly invalid outputs.

An important nuance is the **interaction between inference-time correction and training**. When the supervisor fixes the model's mistakes at inference time, the model "gets away with" producing invalid intermediates. This is why the constraint-aware training loss is important: it gives the model gradient signal about chemical validity during learning, so that over time the supervisor has fewer violations to correct. The curriculum schedule is essential here because the denoising and constraint objectives operate at different scales and the model needs to learn basic feature reconstruction before it can meaningfully respond to valency penalties.

The runtime overhead of the supervisor (3–4×) is moderate and could be reduced with batched constraint checking or by running the Z3 solver only on suspicious atoms (those with high predicted bond counts). For the small molecules tested here the overhead is negligible in absolute terms.

8 Limitations and Future Work

Training data scale. The training experiments use only three molecules (water, methane, ethanol). A serious evaluation would require training on a standard dataset such as QM9 [5] or ZINC [6], parsed from SMILES strings via RDKit.

Molecular complexity. The current system handles molecules up to approximately 15 atoms. The bond head's pairwise computation is $O(n^2)$ in the number of atoms, which becomes expensive for larger molecules. There are no rules for ring structures, stereochemistry, or aromaticity.

Full conservation. With random GNN weights, mass and atom-count conservation cannot be achieved because the model predicts arbitrary element types. Trained weights and a larger dataset are prerequisites for full-conservation validity.

Correction sophistication. The current repair heuristics (Table 4) are greedy and local: they fix one atom at a time without considering the global molecular structure. A more principled approach would be to ask the Z3 solver to find a minimal set of edits that restores validity, treating the correction itself as a constraint-satisfaction problem.

Benchmark scale. The current benchmark uses 50 seeds across 3 reactions. Scaling to approximately 1,000 reactions across diverse reaction types would provide stronger statistical evidence.

Comparison to existing methods. This work does not compare against established molecular generation baselines (e.g., DiGress, GDSS, or MolGAN). Such comparisons would be necessary to position ChemCS relative to the state of the art.

9 Conclusion

This paper presented ChemCS, a system that integrates constraint satisfaction directly into the reverse-diffusion loop of a graph neural network molecular generator. By checking chemical axioms—valency, mass conservation, charge balance, and atom counts—at every denoising step, the supervisor achieves 100% valency validity on a benchmark of three representative reactions, compared with approximately 15% for the same model without supervision. A companion training pipeline adds differentiable constraint penalties through a curriculum schedule, providing gradient signal that complements the inference-time checks.

The architecture demonstrates that combining symbolic reasoning (via Z3) with neural generation (via a GNN denoiser) can enforce domain constraints without sacrificing the flexibility of the generative model. The code, figures, and all 118 tests are available in the project repository. The NumPy inference path runs on CPU; PyTorch is required only for training.

References

[1] E. Hoogeboom, V. G. Satorras, C. Vignac, and M. Welling, "Equivariant diffusion for molecule generation in 3D," in *Proc. ICML*, 2022.

[2] C. Vignac, I. Krawczuk, A. Siraudin, B. Wang, V. Cevher, and P. Frossard, "DiGress: Discrete denoising diffusion for graph generation," in *Proc. ICLR*, 2023.

[3] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *Proc. ICML*, 2021.

[4] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. TACAS*, 2008.

[5] R. Ramakrishnan, P. O. Dral, M. Rupp, and O. A. von Lilienfeld, "Quantum chemistry structures and properties of 134 kilo molecules," *Scientific Data*, vol. 1, 2014.

[6] J. J. Irwin, T. Sterling, M. M. Mysinger, E. S. Bolstad, and R. G. Coleman, "ZINC: A free tool to discover chemistry for biology," *J. Chem. Inf. Model.*, vol. 52, no. 7, 2012.